Smarty Weather Station

Final Project Documentation For ELET2415 Submission

Prepared date: April 21, 2026

ESP32 weather and soil monitoring system with Flask, MongoDB, and Vue dashboard

Prepared for ELET2415 project submission

Prepared date: April 21, 2026

Project folder: `ELET2415/Project`

Document Control

Item	Detail
Project name	Smarty Weather Station
System type	ESP32 IoT weather and soil monitoring station
Main hardware	ESP32 sensor board, ESP32-2432S028 touch gateway board, DHT22, BMP280, capacitive soil sensor
Main software	Arduino firmware, Flask backend, MongoDB, Vue/Vuetify frontend
Main communication path	Sensors -> ESP32 sensor node -> ESP-NOW -> ESP32 display/gateway -> Wi-Fi HTTP -> Flask -> MongoDB -> Vue dashboard
Primary source folders	hardware/, backend/, frontend/, presentation/assets/images/, Project Documentation/
Submission document	This final consolidated documentation file
Git repository	https://github.com/RICK-MONK/Weather-Station

Table Of Contents

1. Executive Summary
2. Project Background And Objectives
3. Requirements And Scope
4. System Architecture
5. Bill Of Materials
6. Component List
7. Hardware Design
8. Schematics And Wiring Diagrams
9. Firmware Design
10. Backend Work
11. Frontend Work
12. Data Model And API Reference
13. Software And Hardware Diagrams
14. Setup And Operation
15. Testing And Validation
16. Results And Project Evidence
17. Troubleshooting, Risks, And Limitations
18. Future Improvements
19. Conclusion
20. Appendices

1. Executive Summary

Smarty Weather Station is an end-to-end IoT environmental monitoring project. It measures weather and soil conditions using ESP32-based hardware, sends readings wirelessly, stores data in a backend database, and displays live and historical readings through a web dashboard.

The project is more than a single-sensor serial monitor demonstration. It combines embedded firmware, sensor validation, ESP-NOW wireless transfer, local TFT display output, Flask API development, MongoDB storage, MQTT publishing support, and a Vue/Vuetify frontend. The final system gives the user both local visibility on the display node and remote visibility through a browser.

The system measures:

- temperature
- relative humidity
- heat index
- atmospheric pressure
- estimated altitude derived from pressure
- soil moisture as a calibrated relative percentage

The strongest engineering features are the system separation between sensor and gateway boards, validation of sensor readings, moving-average smoothing, last-valid-sample fallback behavior, backend timestamping, historical storage, dashboard trend views, and sensor-health flags.

2. Project Background And Objectives

Environmental monitoring is useful in agriculture, greenhouses, classrooms, laboratories, and smart-site monitoring. Many low-cost student sensor projects only display values locally or through the serial monitor. That makes the project useful for a proof of concept, but weak as a complete monitoring system.

This project addresses that gap by building a full sensing-to-dashboard pipeline. The goal is to collect data from multiple sensors, validate the data, move it wirelessly, store it centrally, and present it in a way that a user can understand.

2.1 Problem Statement

Environmental data is only useful if it can be trusted, accessed, and interpreted. A practical weather station must handle noisy readings, temporary sensor failure, wireless communication issues, backend storage, and user-facing visualization.

2.2 Project Objectives

- Measure temperature and humidity with a DHT22 sensor.
- Measure atmospheric pressure with a BMP280 sensor.
- Estimate altitude from BMP280 pressure and a sea-level pressure reference.
- Measure soil moisture using an analog soil moisture sensor.
- Transfer readings between boards using ESP-NOW.
- Forward readings to a Flask backend over Wi-Fi using HTTP.
- Store readings in MongoDB with server-side timestamps.
- Display live readings locally on a TFT display.
- Display live, recent, and analyzed readings in a Vue dashboard.
- Expose backend status and control actions through a frontend control page.

3. Requirements And Scope

3.1 Functional Requirements

Requirement	Implementation

Collect temperature	DHT22 connected to ESP32 sensor node
Collect humidity	DHT22 connected to ESP32 sensor node
Calculate heat index	Firmware computes heat index from temperature and humidity
Collect pressure	BMP280 connected by I2C
Estimate altitude	Firmware derives altitude from pressure using SEA_LEVEL_PRESSURE_HPA
Collect soil moisture	Analog soil sensor connected to ADC GPIO
Transfer data locally	ESP-NOW packet from sensor node to display/gateway node
Display local readings	TFT display on gateway board
Forward to backend	HTTP POST to /api/weather/update
Store history	MongoDB collection configured by backend environment
Show dashboard	Vue/Vuetify frontend with dashboard, live, control, and analysis pages
Analyze data	Backend returns min, max, average, samples, and recent rows

3.2 Non-Functional Requirements

- The system should continue operating when one sensor temporarily fails.
- The dashboard should report stale or delayed data rather than silently hiding it.
- The backend should validate incoming data before storage.
- Sensor data should be stored with consistent backend timestamps.
- The frontend should support metric and imperial display modes.

3.3 In Scope

- ESP32 firmware for sensing and display/gateway operation.
- Weather payload validation and storage.
- Live dashboard views.
- Recent-history queries.
- Date-range analysis queries.
- Backend-tracked control actions.
- Project schematics, wiring references, dashboard screenshots, and visual evidence from the project folder.

3.4 Out Of Scope For The Current Repository

- User authentication.
- Secure device provisioning.
- Weatherproof enclosure.
- Battery or solar power management.
- Device-side execution of backend queued commands.
- Formal automated regression test suite.

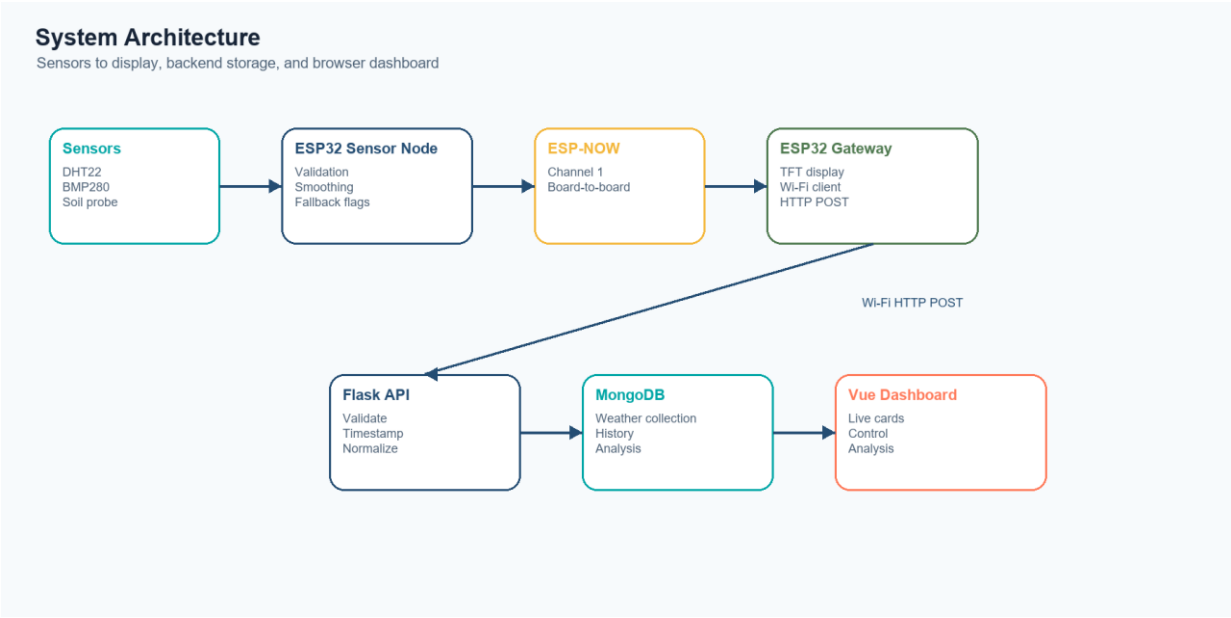
4. System Architecture

The project uses a layered IoT architecture:

- sensing layer
- local wireless transport layer

- gateway and display layer
- backend storage and API layer
- browser dashboard layer

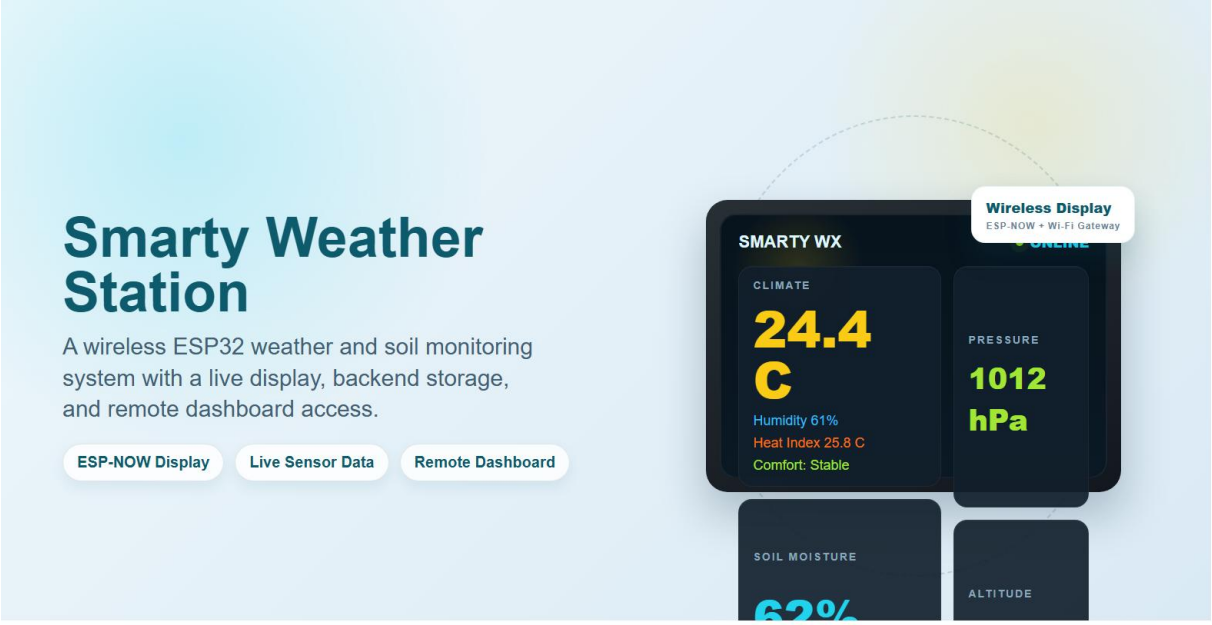
4.1 High-Level Architecture Diagram



Generated system architecture diagram

```
flowchart LR
    DHT[DHT22 temperature and humidity]
    BMP[BMP280 pressure]
    SOIL[Soil moisture sensor]
    SENSOR[ESP32 sensor node]
    ESPNOW[ESP-NOW wireless link]
    GATEWAY[ESP32 display and gateway]
    WIFI[Wi-Fi and HTTP]
    API[Flask API]
    DB[MongoDB]
    UI[Vue dashboard]

    DHT --> SENSOR
    BMP --> SENSOR
    SOIL --> SENSOR
    SENSOR --> ESPNOW
    ESPNOW --> GATEWAY
    GATEWAY --> WIFI
    WIFI --> API
    API --> DB
    API --> UI
```



Wireless weather station overview

4.2 End-To-End Data Flow

21. The sensor ESP32 reads the DHT22, BMP280, and soil sensor.
22. Firmware validates readings and rejects values outside expected ranges.
23. Firmware smooths accepted values using a moving average.
24. Firmware creates a `SensorData` payload.
25. The sensor node sends the payload to the gateway node through ESP-NOW.
26. The gateway displays readings on the TFT screen.
27. The gateway posts JSON to the Flask backend.
28. The Flask backend validates and normalizes the payload.
29. The backend stores the reading in MongoDB and attempts MQTT publishing.
30. The Vue frontend polls backend routes and updates dashboards, charts, and analysis views.

4.3 Communication Paths

Source	Destination	Protocol	Purpose
Sensor node	Display/gateway node	ESP-NOW	Low-overhead local wireless data transfer
Gateway node	Flask backend	HTTP POST over Wi-Fi	Store latest reading
Frontend	Flask backend	HTTP GET/POST	Retrieve data and submit control actions
Backend	MQTT broker	MQTT publish	Optional secondary publish path after database save

4.4 Active Operating Modes

Mode	Description	Repository path
PCB split system	Main submission architecture: sensor board sends ESP-NOW packets to	hardware/active/pcb_sensor_test/pcb_sensor_project/ and hardware/active/pcb_display_project/

	display/gateway board	
Generic ESP-NOW split system	General two-board sender/gateway sketches	hardware/active/espnow_weather/
Standalone firmware path	Single-folder weather station firmware path	hardware/active/weather_station/
Direct Wi-Fi sensor mode	Sensor can post directly to backend when configured	SENSOR_TRANSPORT_DIRECT_WIFI compile-time flag

For the current PCB sensor sketch, `SENSOR_TRANSPORT_DIRECT_WIFI` is set to `0`, so the active PCB path uses ESP-NOW to the display/gateway board.

The bill of materials below includes estimated hardware costs in USD based on representative online pricing checked on April 24, 2026. These figures are estimates for documentation purposes and exclude shipping and tax.

No purchase receipts or confirmed pricing data were found in the project folder. The bill of materials below therefore lists the confirmed hardware quantity, purpose, and interface based on the sketches, documentation, and schematic images in the repository.

m	Quantity	Estimated cost (USD)	Purpose	Interface / Notes
P32 development ard (sensor node)	1	7.69	Sensing and packet transmission	Wi-Fi, ESP-NOW, GPIO, ADC, I2C
P32-2432S028 touch play/gateway board	1	13.99	Local display, touch UI, and backend forwarding	Integrated ESP32 and touch TFT used by the gateway firmware
T22 temperature and midity sensor	1	8.90	Measures ambient temperature and relative humidity	Digital signal on GPIO 4
P280 pressure sensor odule	1	9.95	Measures pressure and supports derived altitude estimate	I2C on GPIO 21 SDA and GPIO 22 SCL
pacitive soil moisture nsor	1	5.90	Measures relative soil moisture	ADC input on GPIO 34
B power leads and all support parts	1 set	5.90	Board power and basic hookup support	Includes USB leads used for programming and demonstration power
ring, headers, and nnectors	1 set	3.95	Sensor and board interconnection	Required for assembly and bench testing
ototype breadboard rdware	1 set	4.95	Temporary assembly and integration	Breadboard or equivalent prototyping hardware
timated hardware total	-	61.23	Estimated total before shipping and tax	Representative online prices checked April 24, 2026

6. Component List

6.1 ESP32 Sensor Node

Role:

- reads the DHT22, BMP280, and soil sensor
- validates and smooths readings
- builds the `SensorData` payload
- sends payload through ESP-NOW in the current PCB configuration

Key firmware file:

- `hardware/active/pcb_sensor_test/pcb_sensor_project/pcb_sensor_project.ino`

6.2 ESP32 Display And Gateway Node

Role:

- receives ESP-NOW packets
- displays live values on the TFT
- evaluates freshness and warning states
- connects to Wi-Fi
- posts weather JSON to Flask

Key firmware file:

- `hardware/active/pcb_display_project/pcb_display_project.ino`

6.3 DHT22 Sensor

Measurements:

- temperature in degrees Celsius
- relative humidity in percent

Used for:

- ambient temperature display
- humidity display
- computed heat index

6.4 BMP280 Sensor

Measurements:

- atmospheric pressure in hPa

Derived value:

- estimated altitude in meters

Important interpretation:

- altitude is calculated from pressure and the configured sea-level reference
- it should not be presented as a directly measured GPS-style altitude

6.5 Soil Moisture Sensor

Measurements:

- raw ADC value
- calibrated relative moisture percentage

Important interpretation:

- the percentage is relative to the firmware calibration constants
- it is not an absolute volumetric water-content measurement

6.6 TFT Display

Purpose:

- local visualization without requiring a browser
- displays weather, soil, system, and connection status screens in the gateway sketch

Current PCB note:

- the PCB display sketch initializes touch-related constants but disables touch navigation with `initializeTouchNavigation()`

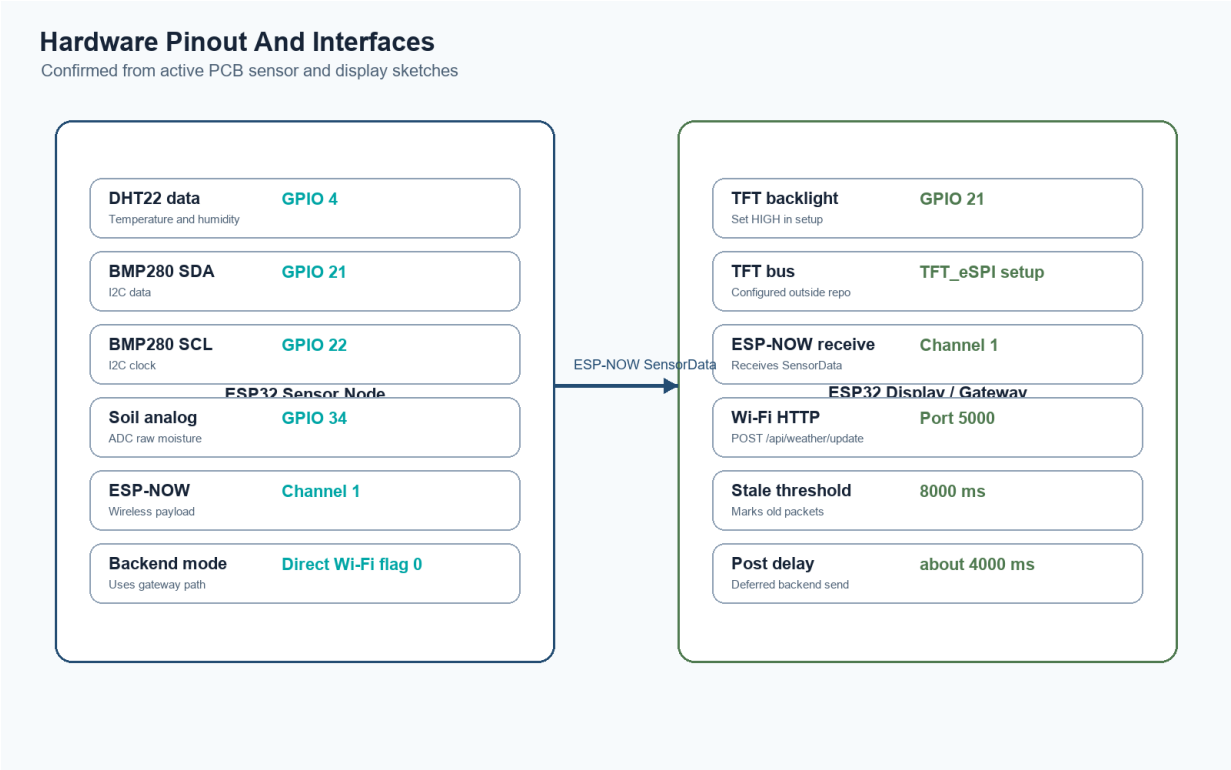
7. Hardware Design

7.1 Hardware Overview

The hardware design separates sensing from display and networking. The sensor node can be positioned near the measurement area, while the gateway node can be positioned where it can display data and reach the backend network.

ESP32 weather station device visual

7.2 Sensor PCB Pin Mapping



Generated hardware pinout diagram

Signal	ESP32 GPIO	Interface	Purpose
DHT22 data	4	Digital	Temperature and humidity
BMP280 SDA	21	I2C data	Pressure sensor data
BMP280 SCL	22	I2C clock	Pressure sensor clock
Soil sensor analog output	34	ADC	Soil moisture raw value

7.3 Display/Gateway Known Signals

Signal	ESP32 GPIO / Setting	Purpose
TFT backlight	GPIO 21	Backlight enabled high in setup
ESP-NOW channel	1	Must align with sender channel
Backend port	5000	Flask API port
Project location label	Project Site	Display header metadata
NTP servers	pool.ntp.org, time.nist.gov	Gateway clock sync

Additional TFT bus pin assignments are controlled by the local `TFT_eSPI` setup outside this repository.

7.4 Firmware Calibration Constants

Constant	Value	Meaning
SOIL_RAW_DRY	720	Dry-side soil calibration reference
SOIL_RAW_WET	160	Wet-side soil calibration reference
SOIL_SAMPLE_COUNT	10	Soil ADC samples averaged per read
SMOOTHING_WINDOW	3	Moving-average sample count

SEA_LEVEL_PRESSURE_HPA	1013.25	Pressure reference for estimated altitude
ESPNOW_WIFI_CHANNEL	1	ESP-NOW channel used by sensor and gateway
DHT_MAX_READ_RETRIES	2	DHT retry count
BMP280_MAX_READ_RETRIES	3	BMP retry count

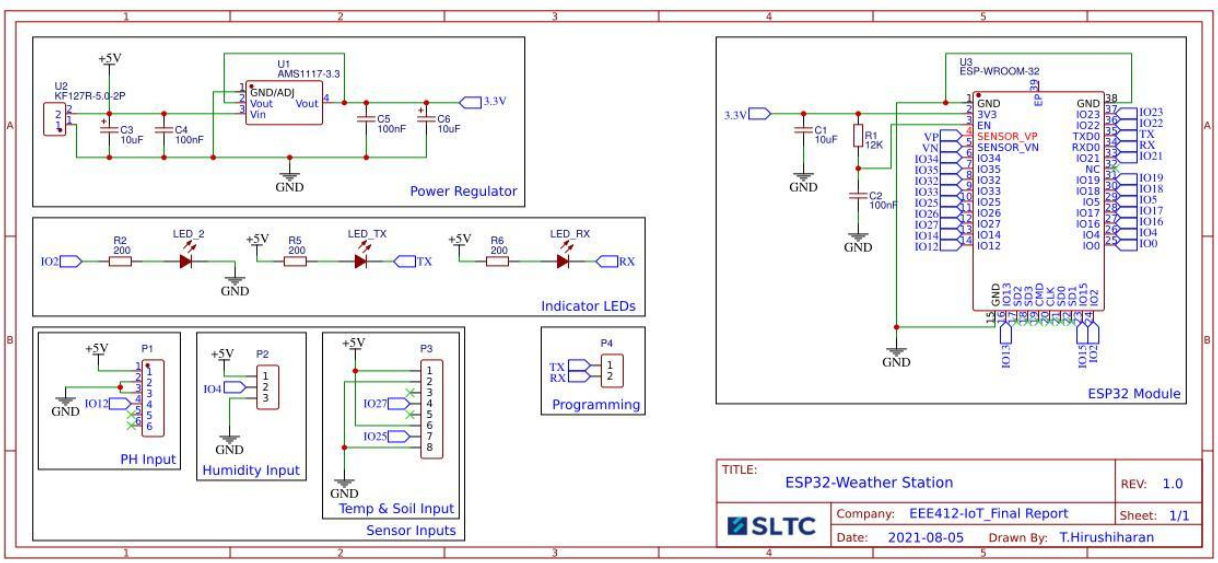
7.5 Sensor Range Validation

Sensor / Field	Firmware validation
Temperature	-40 to 80 degrees Celsius
Humidity	0 to 100 percent
Pressure	300 to 1100 hPa
Estimated altitude	-150 to 4000 m
Soil raw ADC	50 to 4000

8. Schematics And Wiring Diagrams

8.1 Project Schematic Reference

The schematic reference in the project folder shows an ESP32 weather station design with power regulation, indicator LEDs, programming header, sensor inputs, and ESP32 module connections.



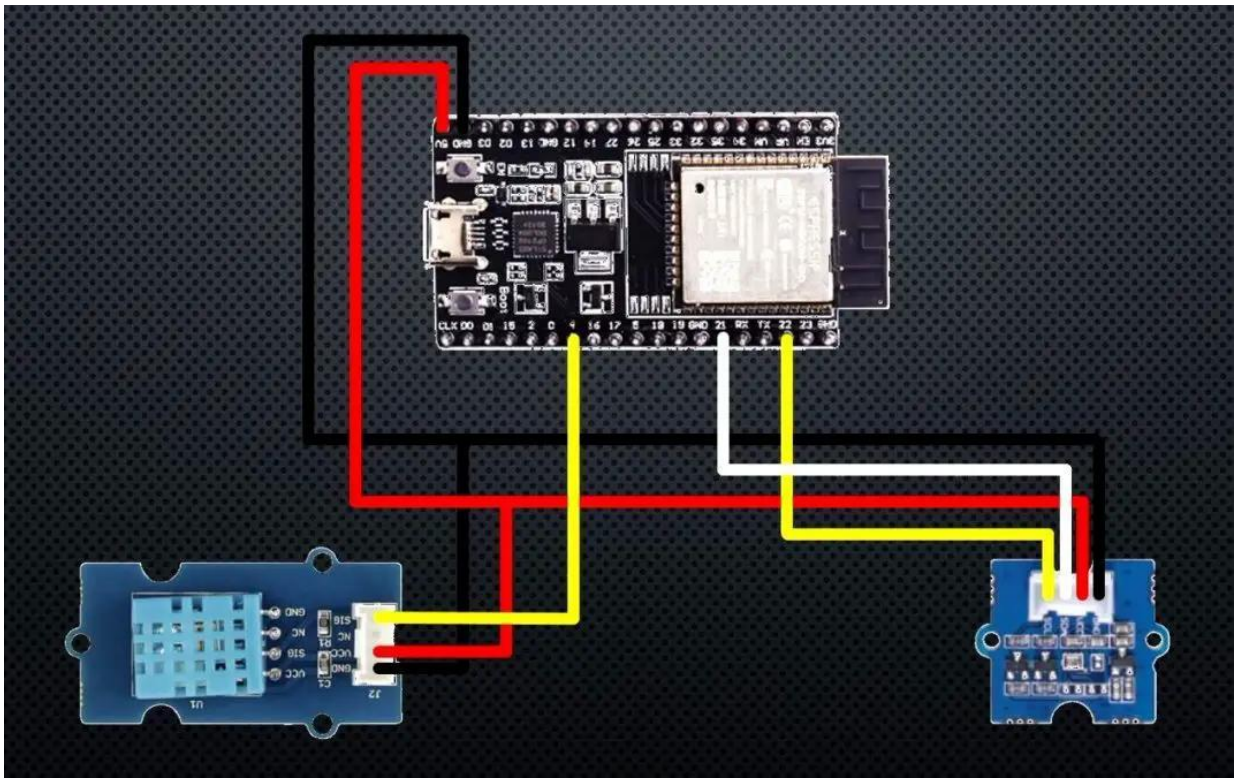
Weather station schematic

Key schematic blocks:

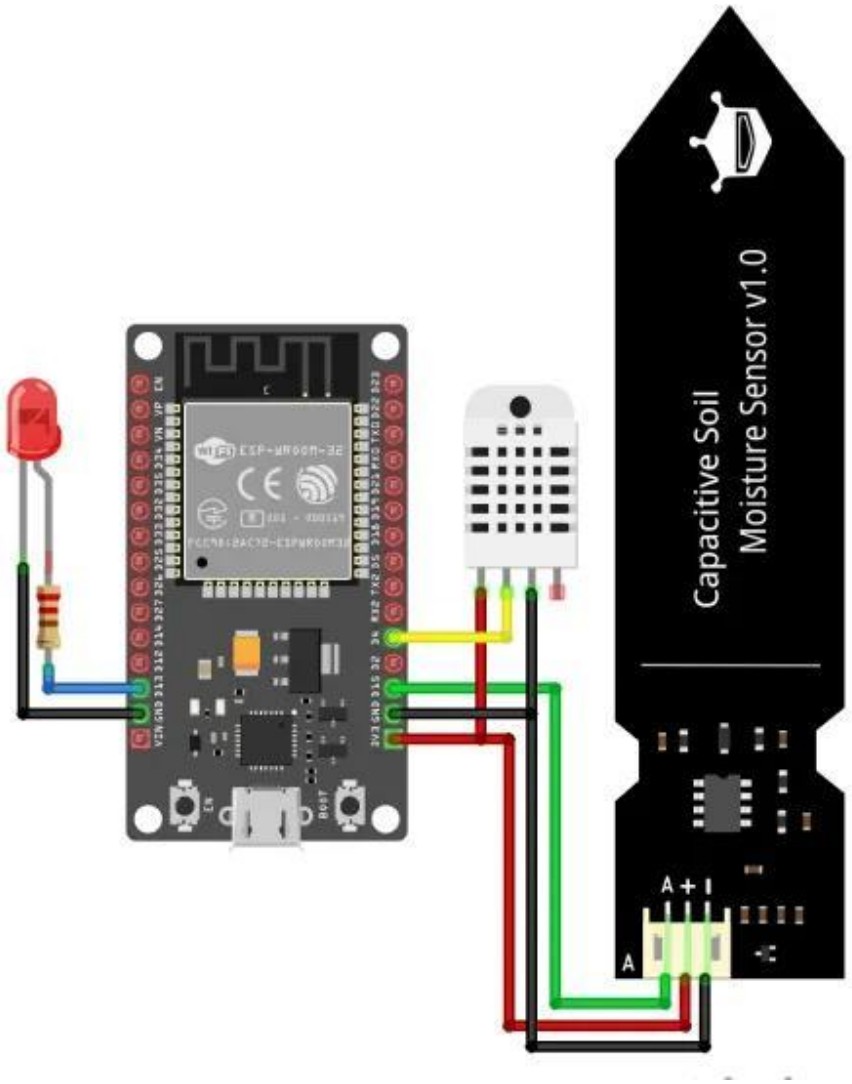
- power regulator from 5 V to 3.3 V
- indicator LEDs
- programming pins
- humidity input
- temperature and soil input
- ESP32 module

8.2 Sensor Wiring Reference

The project includes wiring references for DHT22, BMP280, and soil sensor connections to an ESP32.



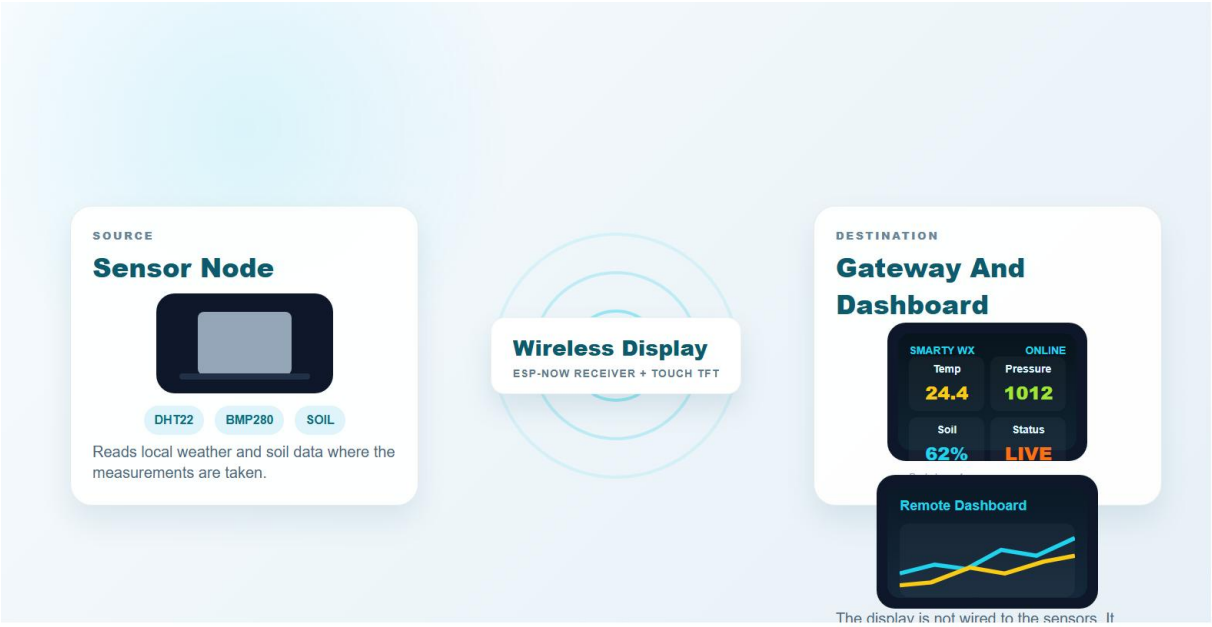
ESP32 DHT22 and BMP280 wiring



ESP32 soil and DHT wiring

8.3 Wireless Link Diagram

The wireless design separates the sensor node and the display/gateway node.



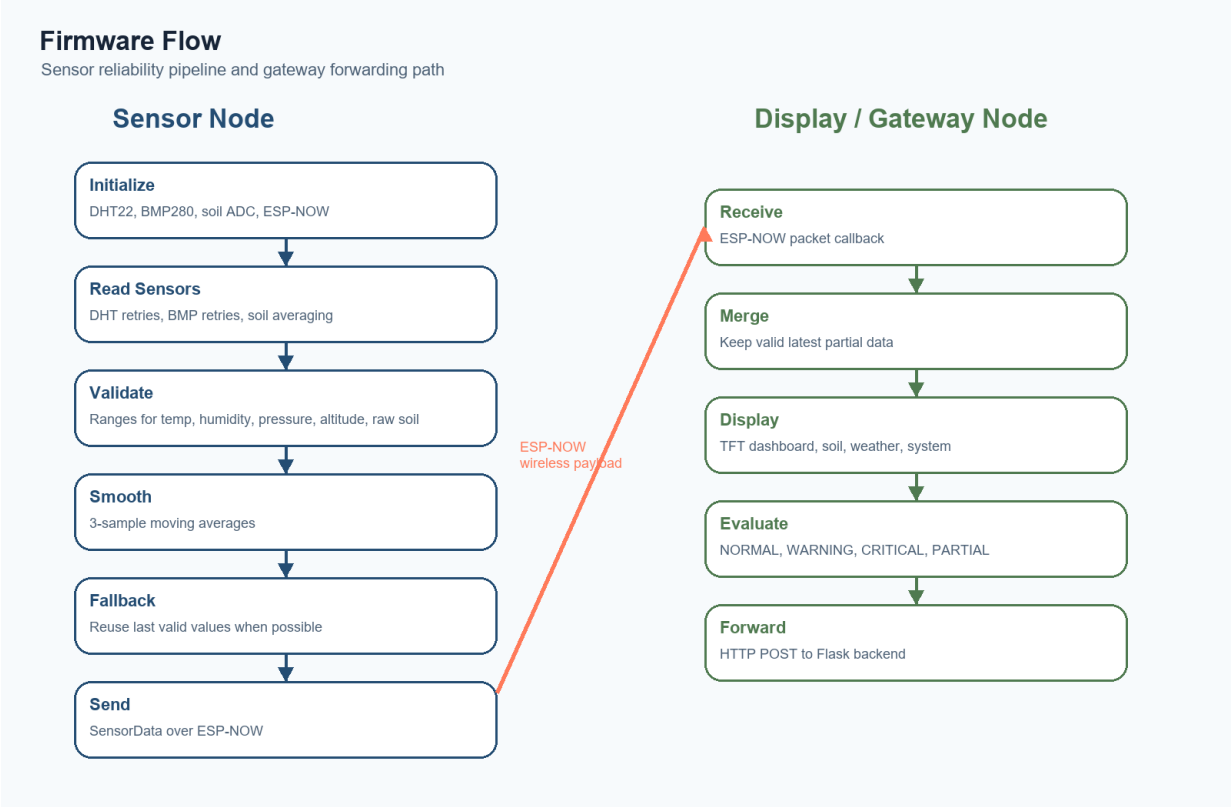
Wireless link concept

8.4 Hardware Block Diagram

```
+------------------+       ESP-NOW        +------------------------+
| ESP32 Sensor PCB | -------------------> | ESP32 Display Gateway  |
|                  |                      |                        |
| DHT22            |                      | TFT display            |
| BMP280           |                      | Wi-Fi client           |
| Soil sensor      |                      | HTTP POST client       |
+------------------+                      +-----------+------------+
                                                      |
                                                      | Wi-Fi HTTP
                                                      v
                                          +--------+---------+
                                          | Flask Backend    |
                                          | MongoDB Storage  |
                                          +--------+---------+
                                                   |
                                                   v
                                          +--------+---------+
                                          | Vue Dashboard    |
                                          +------------------+
```

9. Firmware Design

The firmware is divided across the sensor node and display/gateway node.



Generated firmware flow diagram

9.1 Firmware Source Files

Firmware path	Purpose
hardware/active/pcb_sensor_test/pcb_sensor_project/pcb_sensor_project.ino	Current PCB sensor-node sketch
hardware/active/pcb_display_project/pcb_display_project.ino	Current PCB display/gateway sketch
hardware/active/espnow_weather/sensor_espnow_weather_sender/sensor_espnow_weather_sender.ino	Generic split sensor sender
hardware/active/espnow_weather/display_espnow_weather_gateway/display_espnow_weather_gateway.ino	Generic split display/gateway
hardware/active/weather_station/weather_station.ino	Standalone weather-station sketch

9.2 Sensor Node Responsibilities

- initialize DHT22, BMP280, soil ADC, Wi-Fi station mode, and ESP-NOW
- read all sensors on a repeated loop
- validate sensor values against safe ranges
- retry DHT22 and BMP280 reads
- average soil ADC samples
- smooth accepted values using a 3-sample moving average
- keep last-valid values when sensors fail temporarily

- compute heat index
- convert soil raw values to relative moisture percentage
- send a `SensorData` struct every loop cycle

9.3 Sensor Startup Flow

```
Start serial logging
Initialize DHT22
Initialize I2C for BMP280
Initialize soil ADC
Try BMP280 address 0x76, then 0x77
Set Wi-Fi station mode
Set ESP-NOW channel
Print sender MAC and gateway MAC
Initialize ESP-NOW
Register gateway peer
Begin repeated sensing loop
```

9.4 SensorData Payload

Field	Type	Meaning
temperature	float	DHT22 temperature
humidity	float	DHT22 relative humidity
heatIndex	float	Calculated heat index
pressure	float	BMP280 pressure in hPa
altitude	float	Pressure-derived altitude estimate
seaLevelPressureHpa	float	Reference pressure used for altitude
soilMoisture	int	Raw soil ADC value inside the ESP-NOW struct
altitudeEstimated	uint8_t	Indicates estimated altitude field
dhtOk	uint8_t	DHT read health flag
bmpOk	uint8_t	BMP read health flag
soilOk	uint8_t	Soil read health flag
sampleMillis	uint32_t	Device millis() sample time

Backend JSON also includes:

- `soilRaw`
- `soilMoisturePercent`
- backend-generated `timestamp`

9.5 Gateway Firmware Responsibilities

- initialize TFT display and backlight
- connect to Wi-Fi
- request NTP clock sync
- initialize ESP-NOW receive mode
- receive `SensorData` packets
- merge partial incoming readings with previous readings
- draw dashboard, soil, weather, and system screens
- detect stale packets after 8000 ms
- post JSON payloads to Flask approximately 4000 ms after packet receipt

9.6 Gateway Status Logic

The display/gateway classifies readings using these concepts:

Status	Trigger example
NORMAL	All sensor classes are present and conditions are stable
WARNING	Dry soil, high humidity, or elevated condition
CRITICAL	Extreme temperature, dangerous heat index, or overflow risk
PARTIAL	One or more sensor classes are offline or fallback-based

9.7 Firmware Reliability Features

- DHT22 retry logic.
- BMP280 retry and reinitialization logic.
- Soil raw-range validation.
- Moving-average smoothing.
- Last-valid-sample fallback behavior.
- Partial payload behavior instead of dropping all telemetry.
- Serial logs for sensor health, payload content, Wi-Fi status, TCP tests, and HTTP response codes.
- Sensor health flags forwarded to backend and frontend.

10. Backend Work

10.1 Backend Purpose

The backend is a Flask API service. It receives weather updates from the gateway or sensor, validates the payload, stores normalized documents in MongoDB, publishes saved readings through MQTT when configured, and exposes routes used by the frontend.

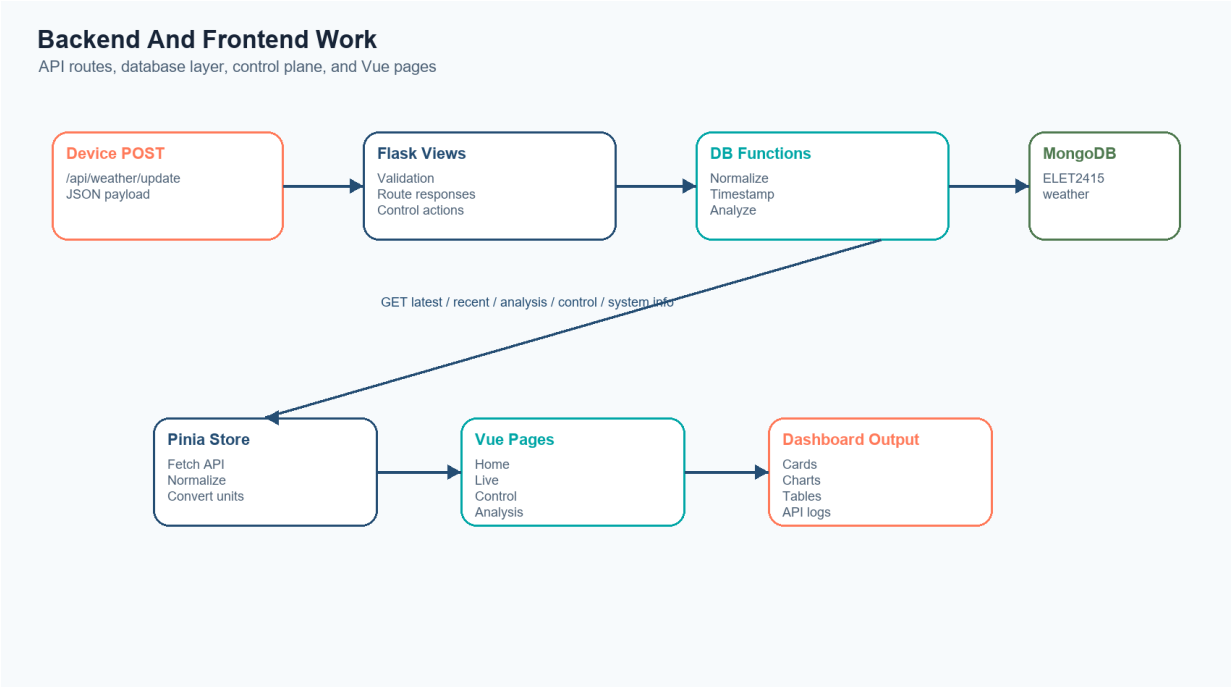
10.2 Backend Stack

Package	Purpose
Flask	API server
Flask-CORS	Allows frontend API calls
PyMongo	MongoDB connection and queries
python-dotenv	Environment variable loading
paho-mqtt	MQTT publishing support

10.3 Backend Source Files

File	Role
backend/run.py	Starts the Flask app
backend/app/__init__.py	App factory and CORS setup
backend/app/views.py	Routes and payload validation
backend/app/functions.py	MongoDB access, normalization, and analysis
backend/app/control.py	In-memory control plane
backend/app/config.py	Environment-based configuration
backend/app/mqtt.py	MQTT connection and publish helper

10.4 Weather Update Handling



Generated backend and frontend flow diagram

`POST /api/weather/update` performs this sequence:

31. Parse JSON body.
32. Check required fields `id` and `type`.
33. Confirm `type` equals `weather`.
34. Confirm optional sensor fields are numeric if present.
35. Normalize values into strings, floats, integers, and boolean flags.
36. Add backend UTC timestamp.
37. Insert document into MongoDB.
38. Attempt MQTT publish to the configured topic.
39. Return save result metadata to the device.

10.5 MongoDB Storage

Default backend database configuration:

Variable	Default
MONGO_HOST	localhost
MONGO_PORT	27017
MONGO_DB	ELET2415
MONGO_COLLECTION	weather

Stored weather documents include the device ID, payload type, timestamp, present sensor values, and health flags.

10.6 Backend Analysis Work

The backend computes analysis for:

- temperature
- humidity
- heat index
- pressure
- altitude
- soil moisture percentage

The analysis route returns:

- count of documents in range
- start and end timestamps
- min, max, and average per metric
- sampled readings for charting
- recent rows for tables

10.7 Backend Control Work

The backend control plane tracks:

- maintenance mode
- last alert acknowledgement
- pending command queue
- recent command history
- delivery mode metadata
- latest weather snapshot metadata

Supported control actions:

- `enable-maintenance`
- `disable-maintenance`
- `acknowledge-alerts`
- `queue-gateway-refresh`
- `queue-gateway-restart`
- `queue-sensor-restart`
- `clear-pending-commands`

Important limitation:

- control actions are backend-tracked only in this repository; the firmware does not currently pull and execute queued commands.

11. Frontend Work

11.1 Frontend Purpose

The frontend is a Vue/Vuetify browser dashboard that displays the current state of the weather station, recent trends, control status, and date-range analysis.

11.2 Frontend Stack

Package	Purpose
Vue 3	UI framework

Vue Router	Page routing
Pinia	Shared state store
Vuetify	UI components
Vite	Development server and build tool
@mdi/font	Icon font dependency

11.3 Frontend Source Files

File	Role
frontend/src/App.vue	App shell and navigation
frontend/src/router/index.js	Route map
frontend/src/store/weather.js	API calls, state, formatting, unit conversion
frontend/src/views/WeatherDashboard.vue	Main dashboard
frontend/src/views/WeatherLive.vue	Live sensor charts
frontend/src/views/WeatherControl.vue	Control and system status
frontend/src/views/WeatherAnalysis.vue	Date-range analysis

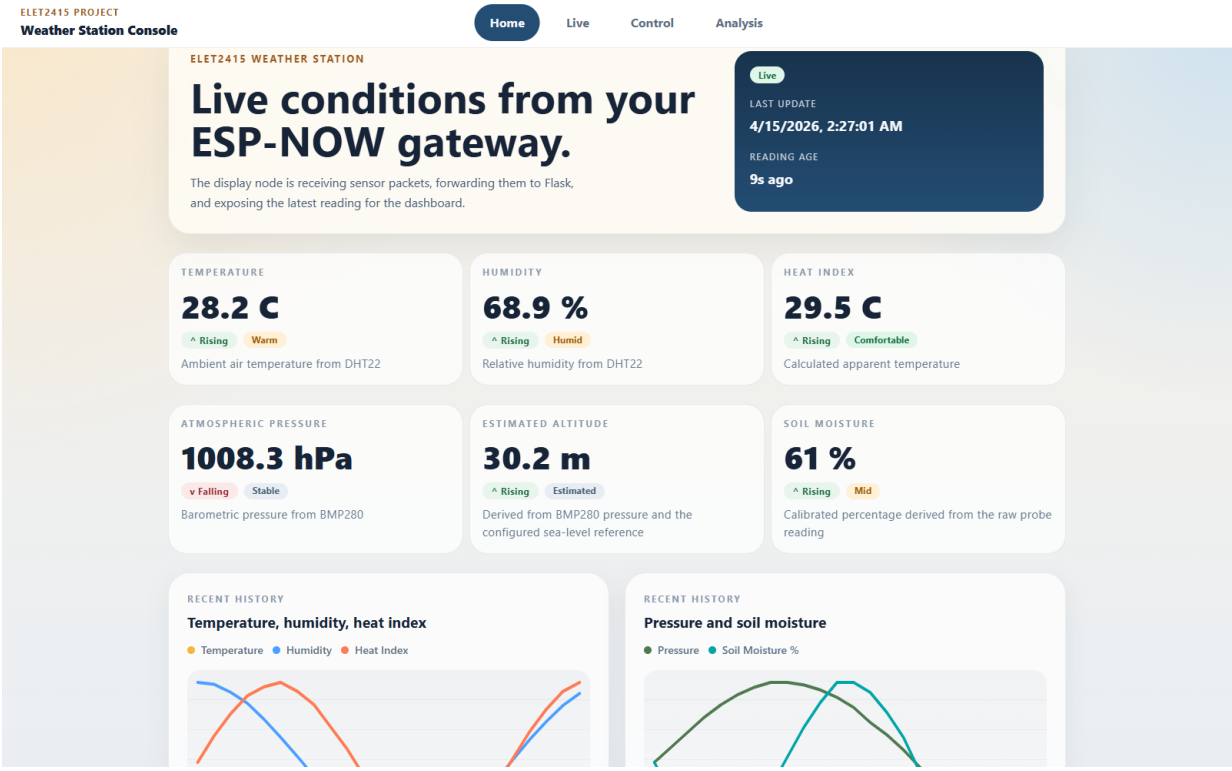
11.4 Route Map

Route	Purpose
/home	Main dashboard
/live/:sensor	Sensor-specific recent trend view
/control	Gateway/system operations view
/analysis/:sensor	Date-range analysis view
/	Redirects to /home

11.5 Dashboard View

The dashboard displays:

- latest reading freshness
- last update time
- reading age
- temperature, humidity, heat index, pressure, altitude, and soil moisture cards
- trend direction versus prior reading
- sensor fallback status
- recent history charts

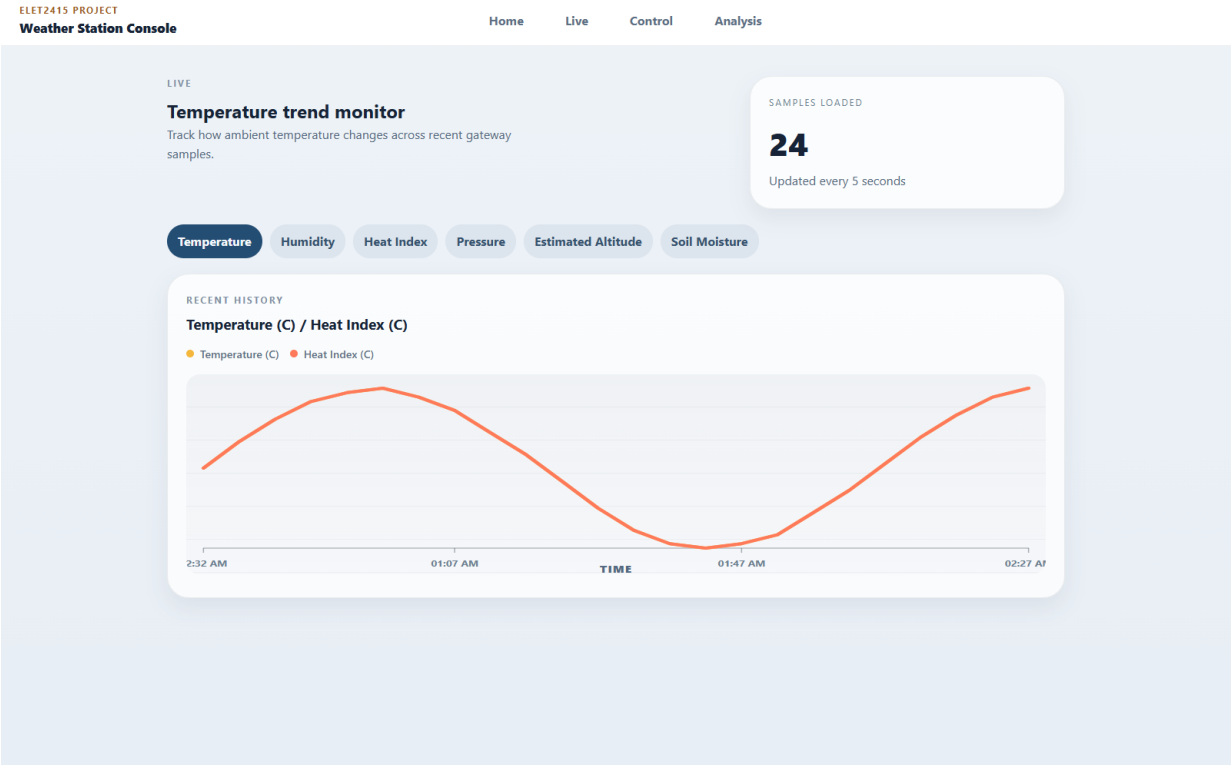


Frontend dashboard screenshot

11.6 Live View

The live view presents recent-history chart pages for:

- temperature
- humidity
- heat index
- pressure
- altitude
- soil moisture

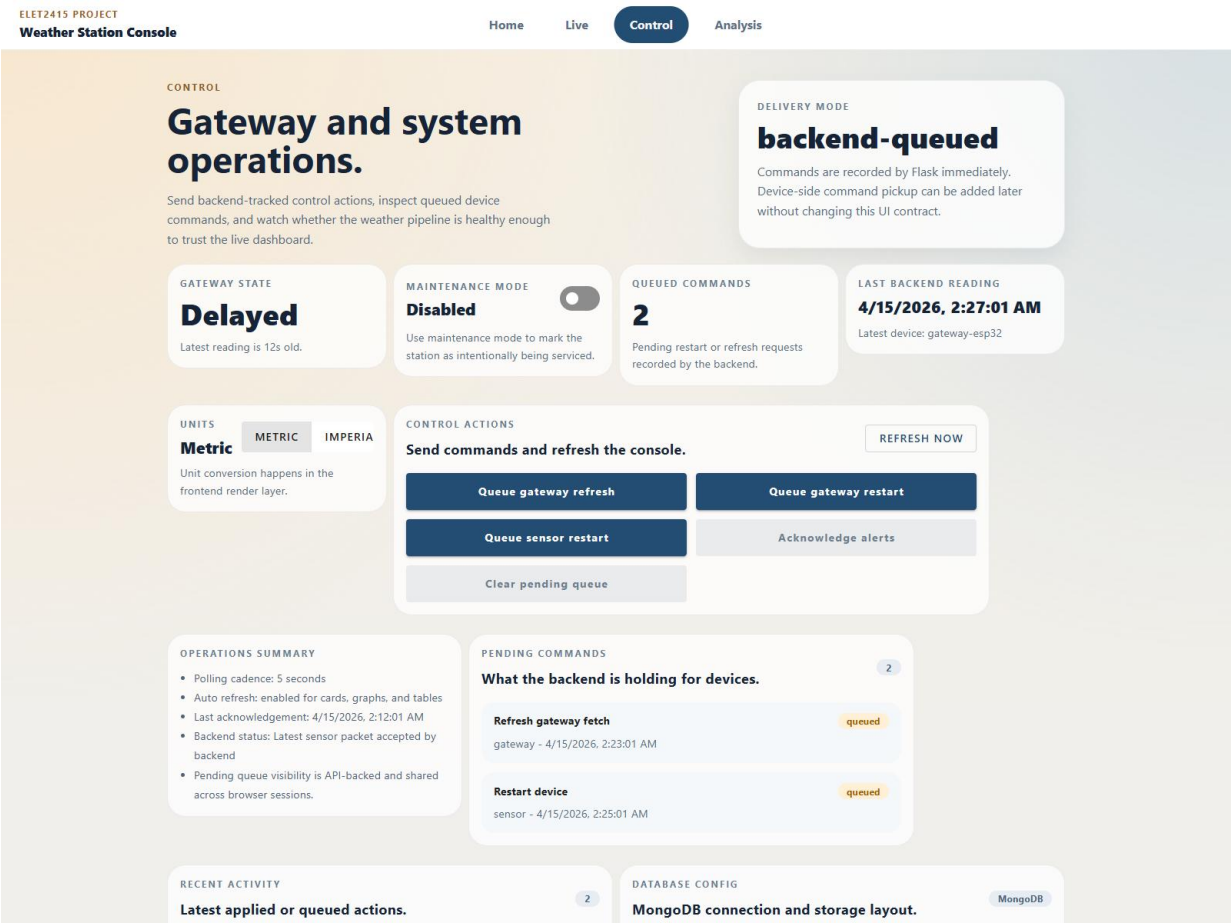


Frontend live screenshot

11.7 Control View

The control page displays:

- gateway health
- maintenance mode
- queued commands
- latest backend reading time
- unit system toggle
- pending commands
- recent control activity
- MongoDB configuration
- schema metadata
- frontend API log entries

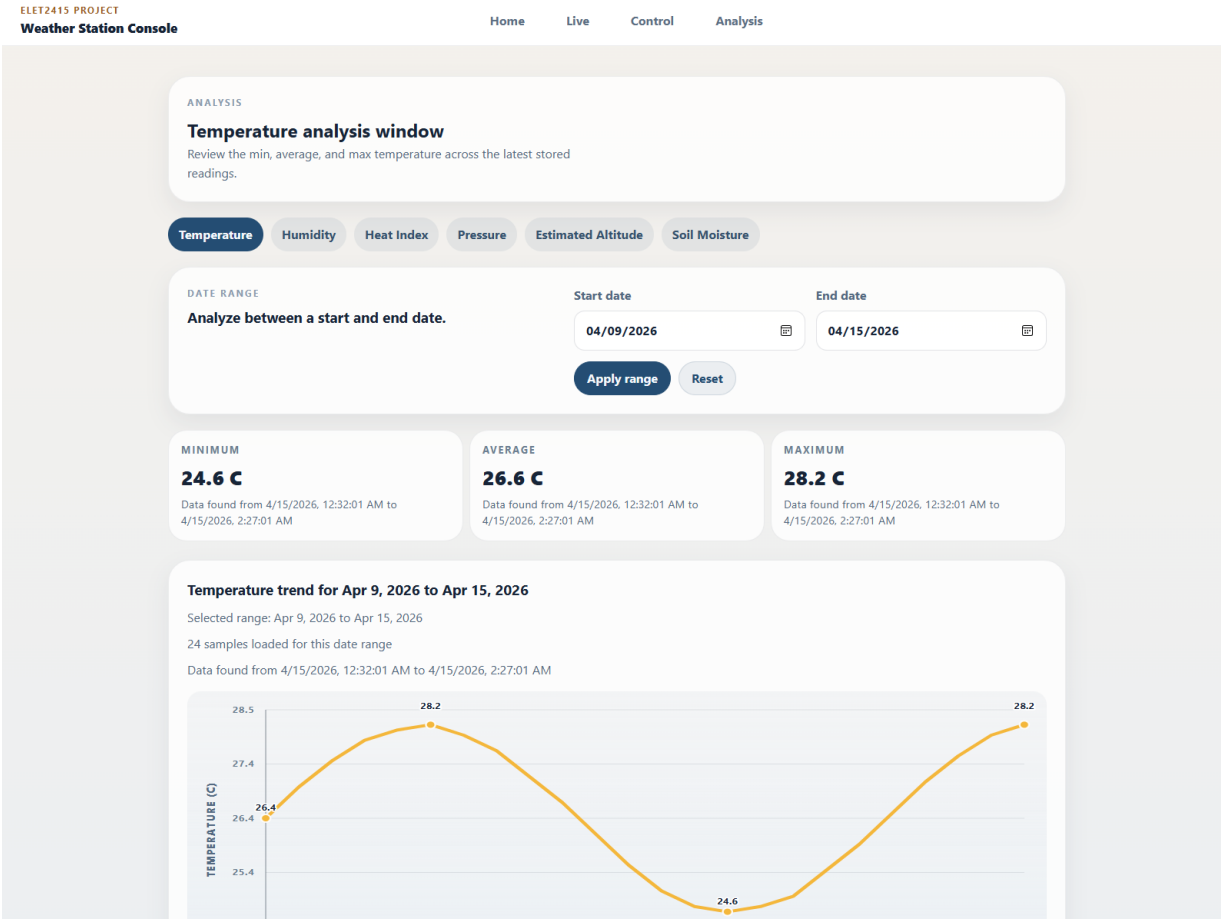


Frontend control screenshot

11.8 Analysis View

The analysis page supports:

- selectable start and end date range
- metric-specific analysis
- min, average, and max display
- sampled chart readings
- recent rows from the selected time range



Frontend analysis screenshot

11.9 Unit Conversion

The frontend supports metric and imperial rendering. Data is stored in the backend in the same normalized units, and display conversion happens in the frontend layer.

Conversions:

- Celsius to Fahrenheit
- hPa to inHg
- meters to feet

12. Data Model And API Reference

12.1 Weather JSON Example

```
{
  "id": "620169874",
  "type": "weather",
  "temperature": 24.6,
  "humidity": 53.1,
  "heatIndex": 24.5,
  "pressure": 932.9,
  "altitude": 691.8,
  "seaLevelPressureHpa": 1013.25,
  "altitudeEstimated": 1,
```

```
  "soilMoisture": 372,
  "soilRaw": 372,
  "soilMoisturePercent": 62,
  "dhtOk": 1,
  "bmpOk": 1,
  "soilOk": 1,
  "sampleMillis": 182340
}
```


12.2 Stored Schema Fields

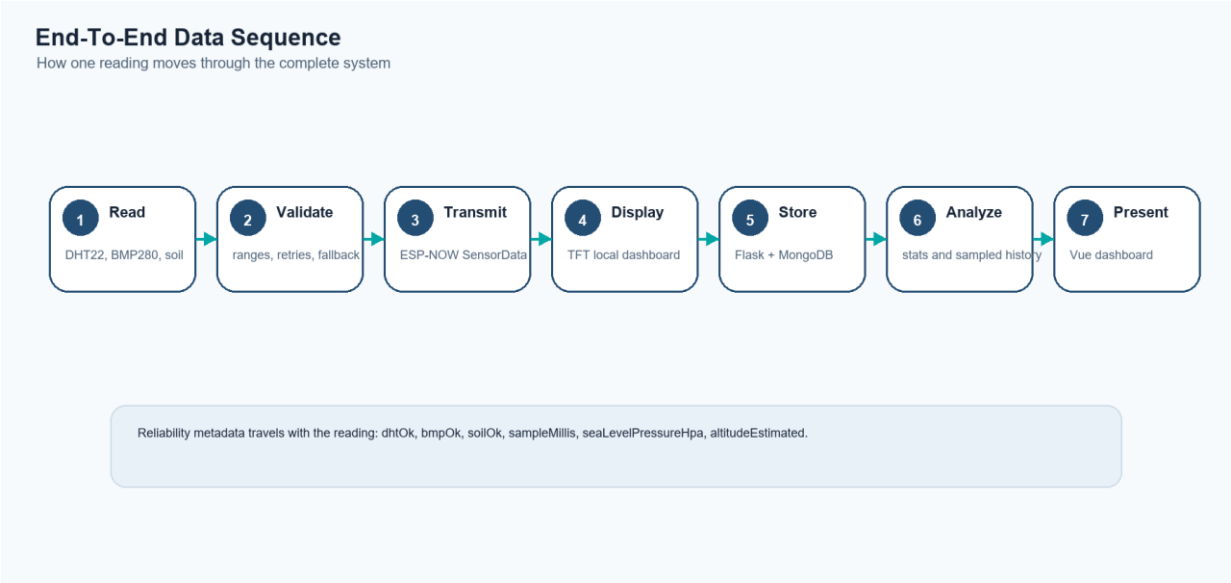
Field	Type	Required	Source
id	string	Yes	Device
type	string	Yes	Device
temperature	float	No	DHT22
humidity	float	No	DHT22
heatIndex	float	No	Firmware calculation
pressure	float	No	BMP280
altitude	float	No	Firmware estimate
soilMoisture	int	No	Device payload
seaLevelPressureHpa	float	No	Firmware constant
altitudeEstimated	bool	No	Firmware flag
soilRaw	int	No	Device payload
soilMoisturePercent	int	No	Firmware calculation
dhtOk	bool	No	Firmware health flag
bmpOk	bool	No	Firmware health flag
soilOk	bool	No	Firmware health flag
sampleMillis	int	No	Device runtime
timestamp	int	Yes	Backend

12.3 API Routes

Method	Path	Purpose
GET	/api/health	Confirm backend is running
POST	/api/weather/update	Submit weather reading
GET	/api/weather/latest	Return latest stored reading
GET	/api/weather/recent?limit=50	Return recent readings
GET	/api/weather/analysis	Return stats and sampled range data
GET	/api/control/status	Return control-plane status
POST	/api/control/action	Submit backend control action
GET	/api/system/info	Return database, schema, and route metadata

13. Software And Hardware Diagrams

13.1 Software Flow Diagram



Generated end-to-end data sequence diagram

```
Sensor firmware
  -> read DHT22, BMP280, soil sensor
  -> validate ranges
  -> smooth values
  -> apply fallback if needed
  -> build SensorData
  -> send ESP-NOW packet

Gateway firmware
  -> receive SensorData
  -> update TFT display
  -> evaluate NORMAL/WARNING/CRITICAL/PARTIAL
  -> connect Wi-Fi
  -> POST JSON to Flask

Backend software
  -> validate JSON
  -> normalize data types
  -> add timestamp
  -> save to MongoDB
  -> publish to MQTT topic if available
  -> expose latest, recent, analysis, control, and system routes

Frontend software
  -> poll backend every 5 seconds
  -> normalize readings
  -> convert units for display
  -> render dashboard, live charts, control, and analysis pages
```

13.2 Backend Process Diagram

```
flowchart TD
    A[HTTP request /api/weather/update] --> B[Parse JSON]
```

```
    B --> C[Validate id and type]
    C --> D[Validate optional numeric fields]
    D --> E[Normalize payload]
    E --> F[Add backend timestamp]
    F --> G[Insert into MongoDB]
    G --> H[Attempt MQTT publish]
    H --> I[Return API response]
```

13.3 Sequence Diagram

```
sequenceDiagram
    participant S as Sensor Node
    participant G as Gateway Node
    participant B as Flask Backend
    participant M as MongoDB
    participant F as Frontend

    S->>S: Read and validate sensors
    S->>G: Send SensorData over ESP-NOW
    G->>G: Update TFT display
    G->>B: POST /api/weather/update
    B->>B: Validate and normalize payload
    B->>M: Insert weather document
    B-->>G: Return save response
    F->>B: Poll latest, recent, analysis, control
    B->>M: Query documents
    B-->>F: Return JSON data
```

13.4 Frontend Page Diagram

```
App.vue
  |
  +-- /home
  |     +-- WeatherDashboard.vue
  |
  +-- /live/:sensor
  |     +-- WeatherLive.vue
  |
  +-- /control
  |     +-- WeatherControl.vue
  |
  +-- /analysis/:sensor
        +-- WeatherAnalysis.vue

Shared data and API layer:
frontend/src/store/weather.js
```

14. Setup And Operation

14.1 Hardware Prerequisites

- ESP32 sensor board
- ESP32 display/gateway board
- DHT22 sensor
- BMP280 sensor
- analog soil moisture sensor
- TFT display

- USB programming cables
- stable power supply

14.2 Software Prerequisites

- Python 3
- Node.js and npm
- MongoDB
- Arduino IDE or `arduino-cli`
- ESP32 board support package
- Arduino libraries:

- DHT sensor library

- Adafruit BMP280 library

- TFT_eSPI

14.3 Backend Setup

From `backend/`:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python run.py
```

The backend listens on `0.0.0.0:5000` by default.

14.4 Frontend Setup

From `frontend/`:

```
npm install
npm run dev
```

The Vite server listens on `0.0.0.0`. By default, the frontend calls:

```
http://<current-browser-host>:5000
```

Set `VITE_API_BASE_URL` if the backend runs somewhere else.

14.5 Firmware Configuration Checklist

Before flashing:

- confirm Wi-Fi SSID and password
- confirm backend IP address constants
- confirm device ID
- confirm gateway MAC address for ESP-NOW mode
- confirm `ESPNOW_WIFI_CHANNEL` is aligned
- confirm `TFT_eSPI` board setup matches the display hardware
- confirm soil calibration values
- confirm BMP280 I2C address

14.6 Recommended Bring-Up Order

40. Start MongoDB.
41. Start the Flask backend.
42. Verify `GET /api/health`.
43. Start the Vue frontend.
44. Open the dashboard in a browser.
45. Flash and power the display/gateway board.
46. Flash and power the sensor board.
47. Watch serial logs on both boards.
48. Confirm backend inserts.
49. Confirm frontend cards and charts update.

15. Testing And Validation

The repository does not include a formal automated test suite. Validation is currently manual and system-level.

15.1 Backend Validation Checklist

- Start Flask successfully.
- Confirm `GET /api/health`.
- Submit a known-good `POST /api/weather/update`.
- Confirm MongoDB insert.
- Confirm `GET /api/weather/latest`.
- Confirm `GET /api/weather/recent`.
- Confirm `GET /api/weather/analysis`.
- Confirm `GET /api/system/info`.
- Confirm each control action route.

15.2 Frontend Validation Checklist

- Dashboard loads without console-breaking errors.
- Latest reading cards populate.
- Recent charts render.
- Unit toggle changes displayed units.
- Control page loads database and schema metadata.
- Analysis page returns min, average, and max for a chosen date range.

15.3 Sensor Node Validation Checklist

- DHT22 starts and returns in-range values.
- BMP280 initializes at address `0x76` or `0x77`.
- Soil sensor raw value is inside the expected ADC range.
- Smoothing produces stable output.
- Fallback messages appear when a sensor is temporarily invalid.
- Sender serial log shows payload activity every loop cycle.

15.4 Gateway Validation Checklist

- Gateway receives ESP-NOW packets.
- Packet age resets on new data.
- TFT display updates correctly.

- Stale status appears if packets stop arriving.
- Backend POST succeeds after packet reception.

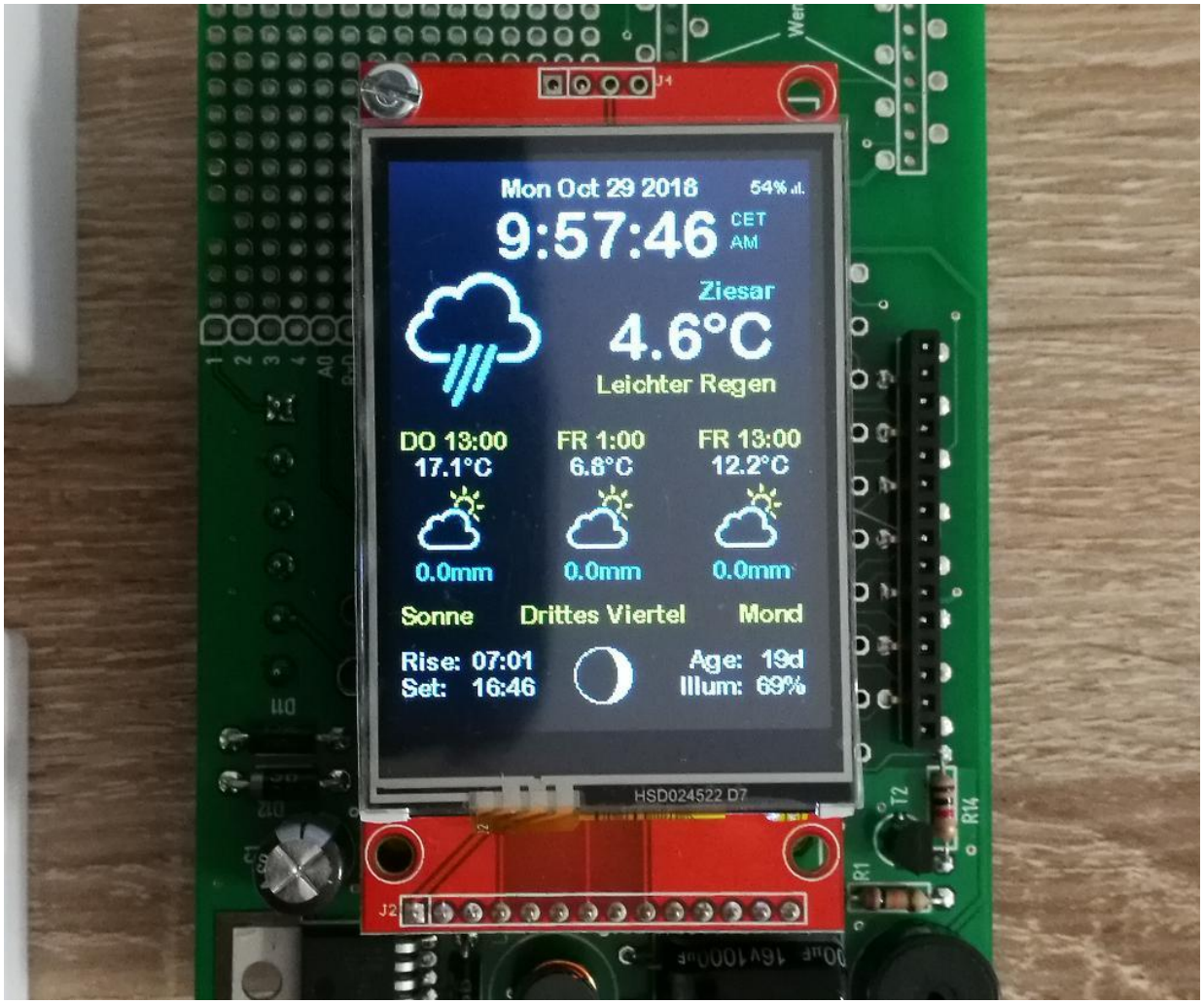
15.5 Integration Test Cases

Test	Method	Expected result
End-to-end happy path	Run sensor, gateway, backend, MongoDB, frontend	Reading appears on TFT, backend, and dashboard
Backend unavailable	Power boards while backend is offline	Local logs show HTTP failure; sensing/display continues
DHT failure	Disconnect or destabilize DHT22	dhtOk false; fallback used if baseline exists
BMP failure	Disconnect BMP280/I2C	bmpOk false; other sensors continue
Soil instability	Disturb or disconnect soil probe	out-of-range protection and fallback behavior
Stale ESP-NOW data	Stop sensor sender	gateway marks packet stale after threshold

16. Results And Project Evidence

16.1 Local Display Evidence

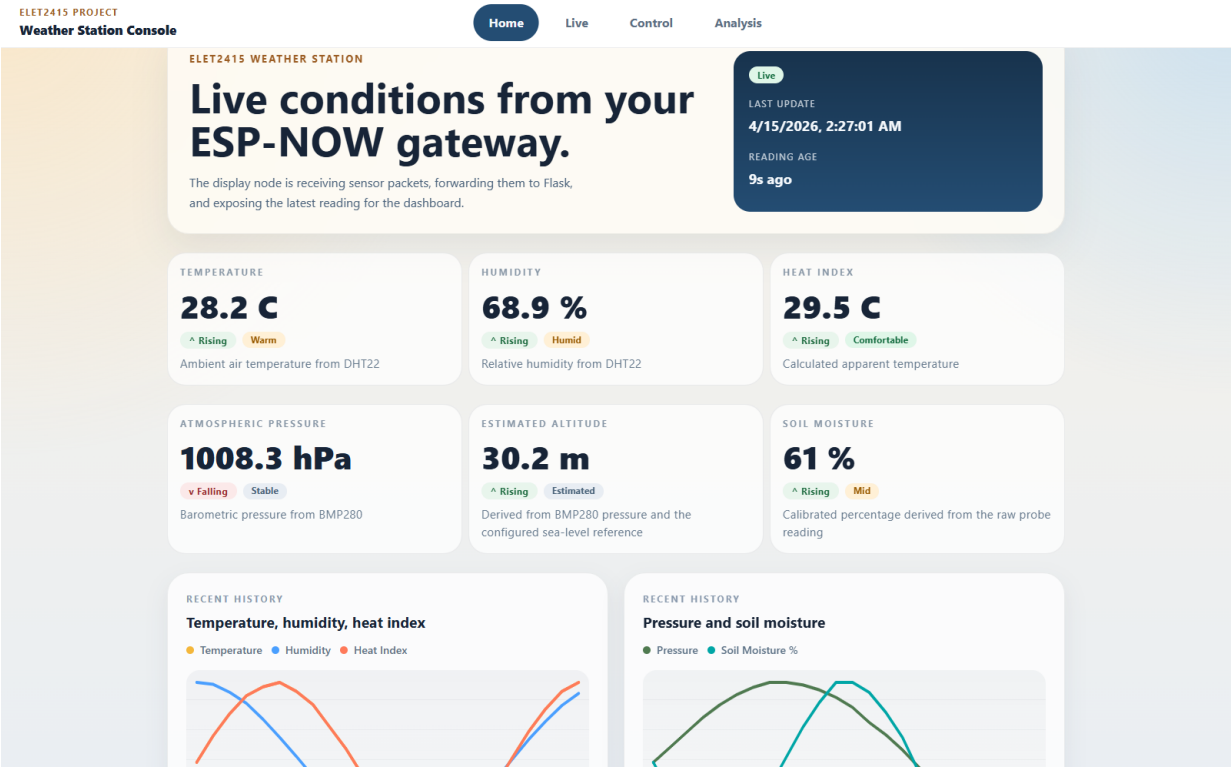
The project folder includes a TFT/display prototype reference image.



Prototype display photo

16.2 Dashboard Evidence

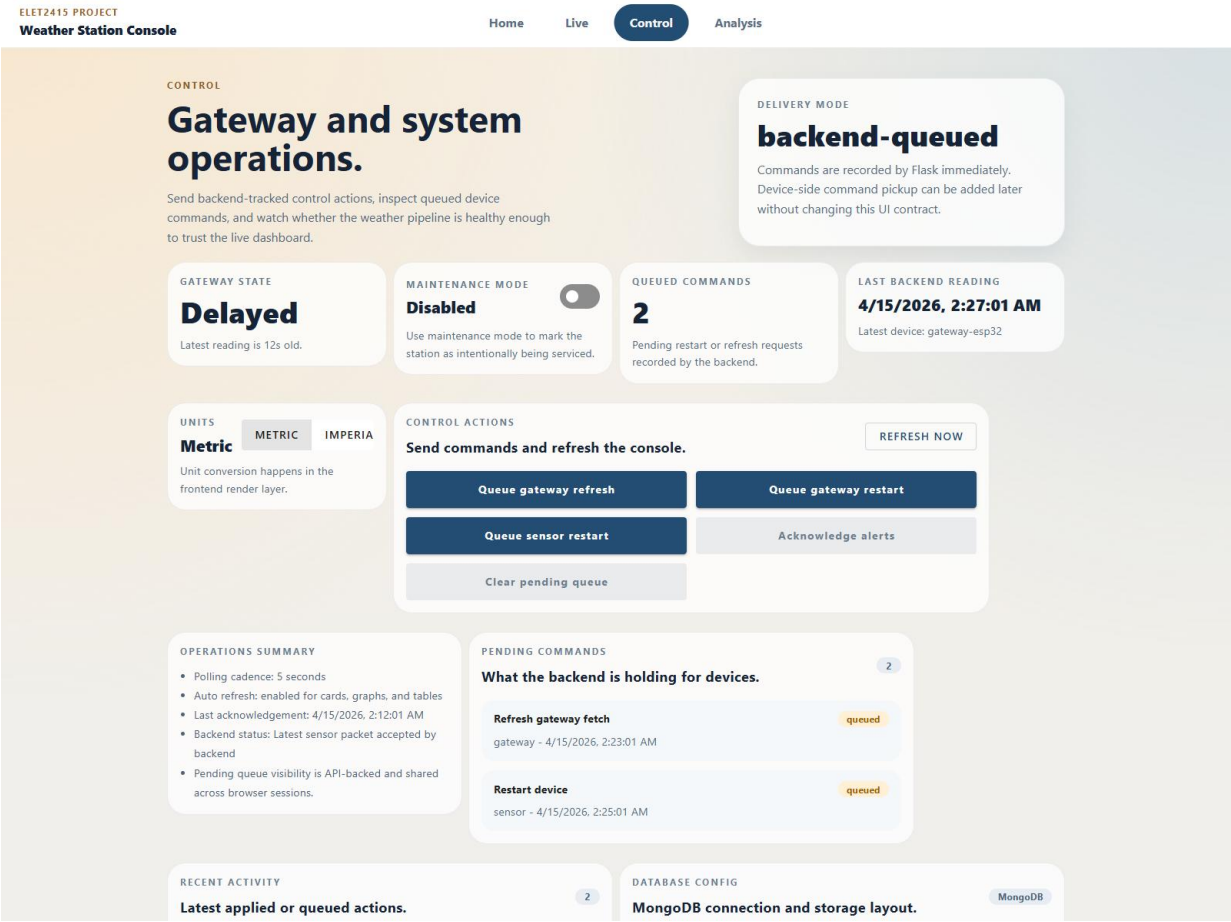
The frontend screenshots show the dashboard receiving current readings, reporting reading age, and displaying recent trend graphs.



Frontend dashboard screenshot

16.3 Control Page Evidence

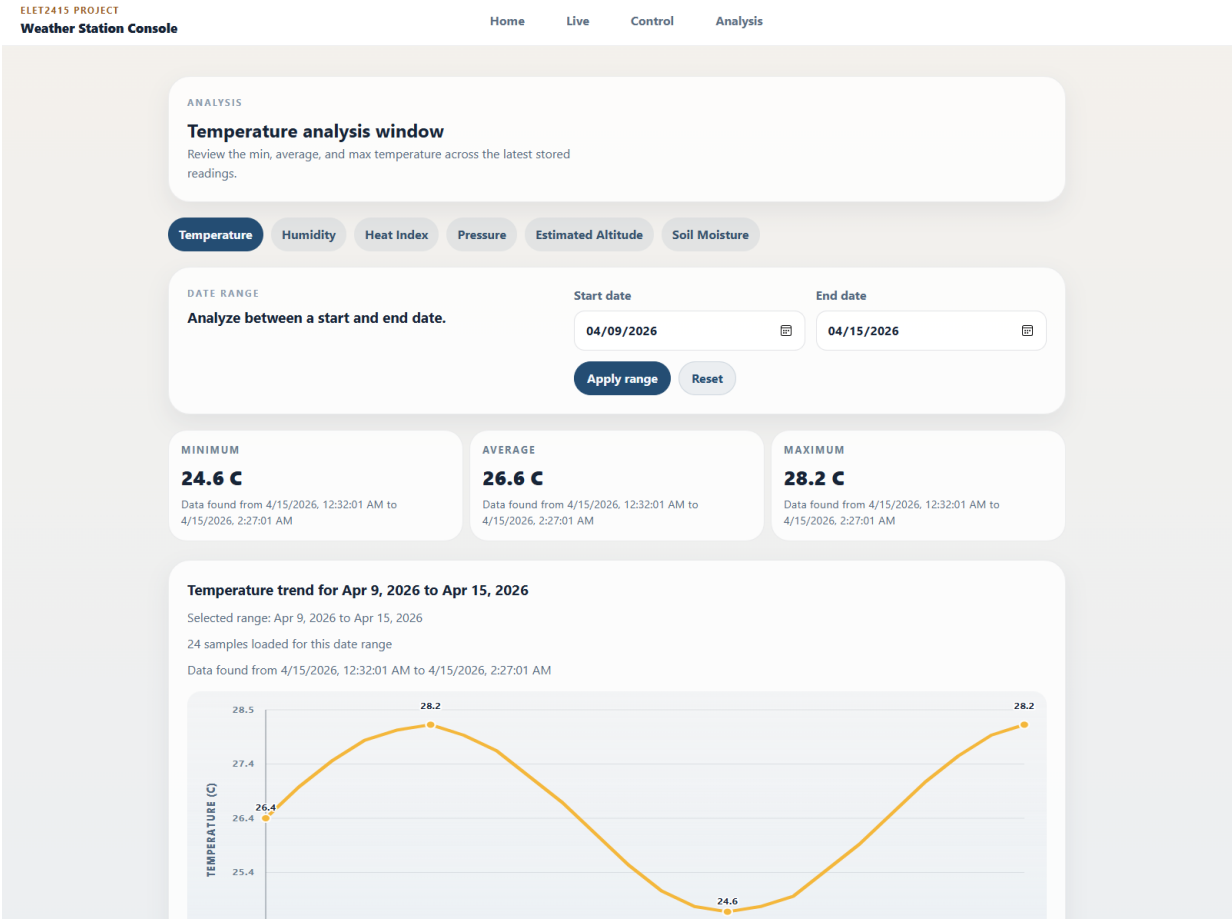
The control page demonstrates backend status, queued commands, database metadata, and system operations.



Frontend control screenshot

16.4 Analysis Evidence

The analysis page demonstrates date-range statistics and sampled readings.

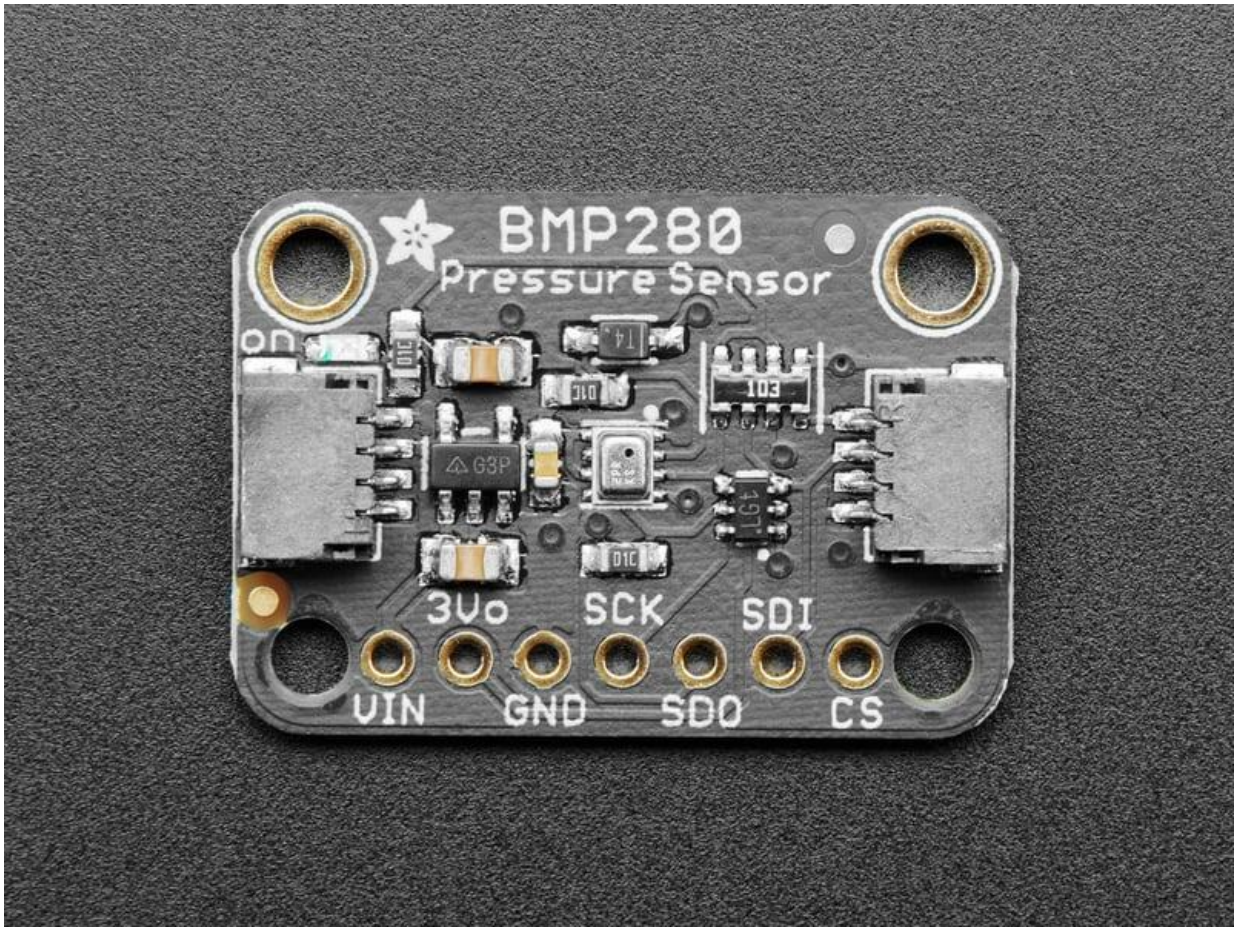


Frontend analysis screenshot

16.5 Component Evidence

[Image could not be inserted: ../presentation/assets/images/esp32-board.jpg;]

[Image could not be inserted: ../presentation/assets/images/dht22.jpg;]



BMP280 sensor

Soil sensor

TFT display

17. Troubleshooting, Risks, And Limitations

17.1 Common Troubleshooting

Problem	Likely causes	Checks
Backend does not receive data	Wrong backend IP, backend offline, Wi-Fi issue, port blocked	Confirm Flask, /api/health, firmware IP, serial logs
ESP-NOW packets not arriving	Channel mismatch, wrong peer MAC, Wi-Fi channel conflict	Confirm ESPNOW_WIFI_CHANNEL, MAC addresses, serial logs
BMP280 fails	Wiring, I2C address, power issue	Check SDA/SCL, addresses 0x76 and 0x77, startup logs
DHT22 read failures	Wiring instability, timing sensitivity	Check data line, supply voltage, retry logs
Soil readings wrong	Calibration mismatch, ADC range issue, probe placement	Inspect raw ADC, recalibrate wet/dry constants
TFT display incorrect	TFT_eSPI setup mismatch	Confirm local TFT driver and pin configuration
Frontend stale or empty	No backend data, wrong API base URL, backend query failed	Check API logs and /api/weather/latest
Control actions do not affect devices	Device-side command consumer not implemented	Treat as known current limitation

17.2 Known Risks

- Firmware network settings are hard-coded and must be updated when the Wi-Fi or backend host changes.
- ESP-NOW and Wi-Fi coexistence depends on channel alignment.
- Control state is stored in backend memory and is lost on backend restart.
- The current API has no authentication.
- Soil moisture is relative and probe-dependent.
- Altitude is estimated from pressure and calibration.
- There is no persistent local queue if backend posting fails.

17.3 Current Limitations

- No automated frontend/backend/firmware test suite in the repository.
- No device-side command pickup for queued backend actions.
- No weatherproof enclosure or power-management design.
- No long-duration endurance test results included.
- TFT bus configuration depends on local `TFT_eSPI` setup outside the repository.

18. Future Improvements

- Add a weatherproof enclosure.
- Add battery or solar power support.
- Persist control state in MongoDB.
- Add device-side polling/execution for queued commands.
- Add backend authentication or LAN access control.
- Add alerting through email, SMS, or push notification.
- Add an automated irrigation actuator.
- Add configuration mode for Wi-Fi and backend address.
- Add automated tests for API routes and frontend store behavior.
- Add long-duration logging and reliability testing.

19. Conclusion

Smarty Weather Station demonstrates a complete IoT monitoring system. It reads multiple environmental sensors, validates and smooths measurements, sends data over ESP-NOW, displays readings locally, forwards data to a Flask backend, stores readings in MongoDB, and visualizes live and historical values in a Vue dashboard.

The project shows integration across hardware, firmware, backend, database, and frontend layers. It also addresses practical engineering concerns such as noisy sensors, partial failures, stale data, backend validation, and user-facing status reporting. The result is a strong project platform that can be extended into smart agriculture, greenhouse monitoring, classroom demonstration, and environmental automation.

20. Appendices

Appendix A: Main Repository Files Used

Path	Use In This Document
README.md	Repository overview
Project Documentation/01-project-overview.md	Project goals and scope
Project Documentation/02-system-architecture.md	Architecture and data lifecycle
Project Documentation/03-hardware-design.md	Components, pin mapping, calibration

Project Documentation/04-firmware-design.md	Firmware structure and behavior
Project Documentation/05-backend-design.md	Backend architecture
Project Documentation/06-frontend-design.md	Frontend architecture
Project Documentation/07-api-reference.md	API routes and schema
Project Documentation/08-setup-and-operation.md	Setup steps
Project Documentation/09-testing-and-validation.md	Validation strategy
Project Documentation/10-troubleshooting-and-risks.md	Troubleshooting and risks
docs/weather-station-report.md	Report draft source
hardware/active/pcb_sensor_test/pcb_sensor_project/pcb_sensor_project.ino	Current sensor firmware constants and behavior
hardware/active/pcb_display_project/pcb_display_project.ino	Current gateway/display firmware constants and behavior
backend/app/views.py	API routes and validation logic
backend/app/functions.py	MongoDB normalization and analysis
frontend/src/store/weather.js	Frontend API, unit conversion, and normalization
frontend/src/views/	Dashboard, live, control, and analysis UI

Appendix B: Image And Diagram Assets Used

Asset	Purpose
presentation/assets/images/wireless-weather-station.png	System overview visual
presentation/assets/images/weather-station-device.png	Display/device visual
presentation/assets/images/wireless-link.png	Wireless display/gateway concept
presentation/assets/images/weather-station-schematic.jpg	Schematic reference
presentation/assets/images/esp32-dht-bmp-wiring.jpg	DHT22 and BMP280 wiring reference
presentation/assets/images/esp32-soil-dht-wiring.jpg	Soil and DHT wiring reference
presentation/assets/images/frontend-home.png	Dashboard screenshot
presentation/assets/images/frontend-live.png	Live view screenshot
presentation/assets/images/frontend-control.png	Control page screenshot
presentation/assets/images/frontend-analysis.png	Analysis page screenshot
presentation/assets/images/prototype-display-photo.jpg	TFT/display prototype photo
presentation/assets/images/esp32-board.jpg	ESP32 component image
presentation/assets/images/dht22.jpg	DHT22 component image
presentation/assets/images/bmp280.jpg	BMP280 component image
presentation/assets/images/soil-sensor.jpg	Soil sensor component image
presentation/assets/images/tft-display.jpg	TFT display component image
Project Documentation/final-documentation-assets/system-architecture-generated.png	Generated final architecture diagram for submission
Project Documentation/final-documentation-assets/hardware-pinout-generated.png	Generated hardware pinout and interface diagram
Project Documentation/final-documentation-assets/firmware-flow-generated.png	Generated firmware flow diagram
Project Documentation/final-documentation-assets/backend-frontend-flow-generated.png	Generated backend/frontend flow diagram

Project Documentation/final-documentation-assets/data-sequence-generated.png	Generated end-to-end sequence diagram

The project folder also contains SVG diagram assets:

- `presentation/assets/images/system-architecture.svg`
- `presentation/assets/images/wiring-diagram.svg`
- `presentation/assets/images/schematic-overview.svg`
- `presentation/assets/images/espnow-flow.svg`
- `presentation/assets/images/backend-flow.svg`
- `presentation/assets/images/data-flow-animated.svg`

Appendix C: Backend Environment Variables

```
FLASK_HOST=0.0.0.0
FLASK_PORT=5000
FLASK_DEBUG=true
MONGO_HOST=localhost
MONGO_PORT=27017
MONGO_DB=ELET2415
MONGO_COLLECTION=weather
MQTT_BROKER=localhost
MQTT_PORT=1883
MQTT_TOPIC=weatherstation/update
```

Appendix D: Submission Notes

- This document is based only on project-folder resources and current source code.
- BOM costs are estimated from representative online component listings checked on April 24, 2026, and they exclude shipping and tax.
- The document describes the PCB split system as the main submission architecture because it matches the active PCB sensor and display sketches.
- Some image assets are project visuals, screenshots, references, or presentation graphics. Actual assessment photos can be added later if more project-build photos are captured.